

Marine Drifter Trajectory Prediction using Physics-Residual Transformer Neural ODE

1 Model 1 Architecture

The proposed framework consists of three major modules:

1. Transformer encoder,
2. Neural ODE latent dynamics block,
3. residual prediction head.

1.1 Overall Framework

The forward process is:

$$X \rightarrow \text{Transformer Encoder} \rightarrow z_0 \rightarrow \text{Neural ODE} \rightarrow z_T \rightarrow \text{MLP Head} \rightarrow \hat{r} \quad (1)$$

Final prediction:

$$\hat{y} = \Delta x_{adv} + \hat{r} \quad (2)$$

where:

- $\hat{r}$ is the predicted residual correction.

2 Transformer Encoder

2.1 Motivation

Marine drifter trajectories exhibit strong temporal dependency and long-range correlation. Traditional recurrent networks may suffer from vanishing gradients and limited temporal receptive fields.

Transformers solve this problem using self-attention mechanisms.

2.2 Positional Encoding

Since Transformers lack inherent temporal ordering, sinusoidal positional encoding is added:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (3)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (4)$$

where:

- pos denotes timestep index,
- d denotes embedding dimension.

2.3 Transformer Layer

Each Transformer encoder layer contains:

- multi-head self-attention,
- feed-forward network,
- residual connection,
- layer normalization.

The attention mechanism computes:

$$Attention(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (5)$$

where:

- Q is query,
- K is key,
- V is value.

The encoder outputs a latent trajectory representation:

$$z_0 \in \mathbb{R}^{128} \quad (6)$$

3 Neural ODE Dynamics Module

3.1 Motivation

Discrete neural networks model dynamics layer-by-layer. However, physical ocean systems evolve continuously in time.

Neural ODEs model latent dynamics as continuous differential equations:

$$\frac{dz(t)}{dt} = f_\theta(z(t), t) \quad (7)$$

where:

- $z(t)$ is latent state,
- f_θ is a neural network parameterization.

The latent trajectory is obtained through ODE integration:

$$z(T) = z(0) + \int_0^T f_\theta(z(t), t) dt \quad (8)$$

3.2 ODE Function

The implemented ODE function uses:

- linear projection,
- Tanh activation,
- residual latent evolution.

This continuous latent evolution helps:

- improve temporal smoothness,
- reduce discontinuity,
- better model ocean flow dynamics.

4 Residual Prediction Head

After latent dynamics evolution, the hidden representation is passed through a multilayer perceptron:

$$\hat{r} = W_2 \sigma(W_1 z + b_1) + b_2 \quad (9)$$

The output dimension is two:

$$\hat{r} = [\hat{r}_{north}, \hat{r}_{east}] \quad (10)$$

representing residual displacement corrections in meters.

5 Loss Function

5.1 Prediction Loss

The training objective primarily minimizes residual prediction error:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \|\hat{r}_i - r_i\|^2 \quad (11)$$

The implementation additionally uses Smooth L1 loss during validation for robustness.

5.2 Physics Regularization

To stabilize residual magnitude, an auxiliary physics regularization term is added:

$$\mathcal{L}_{phys} = \frac{1}{N} \sum_{i=1}^N \|\hat{r}_i\|^2 \quad (12)$$

The final objective becomes:

$$\mathcal{L} = \mathcal{L}_{MSE} + \lambda \mathcal{L}_{phys} \quad (13)$$

where:

$$\lambda = 0.01 \tag{14}$$

controls the regularization strength.

6 Training Strategy

6.1 Optimization

The framework uses AdamW optimization:

- learning rate: 10^{-4} ,
- weight decay: 10^{-5} .

AdamW improves generalization by decoupling weight decay from adaptive optimization.

6.2 Learning Rate Scheduling

A ReduceLROnPlateau scheduler is used:

- factor: 0.5,
- patience: 3 epochs.

This strategy reduces learning rate when validation performance stagnates.

6.3 Checkpoint Mechanism

To prevent training interruption and support long training runs, checkpoints are periodically saved.

The framework automatically:

- detects latest checkpoint,
- resumes training,
- restores optimizer state.

This design is especially useful for large-scale ocean trajectory experiments.

7 Evaluation Metrics

7.1 RMSE in Geographic Space

Latitude RMSE:

$$RMSE_{lat} = \sqrt{\frac{1}{N} \sum_i (\hat{y}_{lat,i} - y_{lat,i})^2} \tag{15}$$

Longitude RMSE:

$$RMSE_{lon} = \sqrt{\frac{1}{N} \sum_i (\hat{y}_{lon,i} - y_{lon,i})^2} \tag{16}$$

7.2 Geodesic Error

To measure real-world trajectory accuracy, Haversine distance is used.

For two geographic coordinates:

$$(\phi_1, \lambda_1), (\phi_2, \lambda_2) \quad (17)$$

The geodesic distance is:

$$d = 2R \arcsin(\sqrt{a}) \quad (18)$$

where:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (19)$$

and $R = 6371$ km is Earth’s radius.

The following statistics are computed:

- mean geodesic error,
- median geodesic error,
- P90 geodesic error.

8 Experimental Results and Analysis

Example evaluation results:

Metric	Value
Latitude RMSE	0.0591
Longitude RMSE	0.7425
Mean Geodesic Error	8.57 km
Median Geodesic Error	5.23 km
P90 Geodesic Error	14.96 km

Table 1: Prediction Performance of the Proposed Model

8.1 Performance Interpretation

The results demonstrate several important characteristics:

1. The physics-advection baseline significantly stabilizes prediction.
2. Residual learning reduces the burden on the neural network.
3. Transformer attention effectively captures temporal dependencies.
4. Neural ODE improves latent trajectory smoothness.
5. Median geodesic error is substantially lower than mean error, indicating that large-error outliers are relatively infrequent.

8.2 Longitude Error Analysis

Longitude RMSE is larger than latitude RMSE. Possible reasons include:

- east-west ocean current variability,
- coordinate scaling sensitivity,
- accumulated advection uncertainty,
- unresolved mesoscale eddies.

The longitude conversion factor:

$$\cos(\phi) \tag{20}$$

may also amplify instability at high latitude regions.

8.3 Advantages of Hybrid Modeling

Compared with purely data-driven methods, the proposed framework offers:

- stronger physical interpretability,
- improved optimization stability,
- lower prediction variance,
- better generalization.

Compared with purely physical methods, the model additionally captures:

- nonlinear residual flow patterns,
- turbulence-induced deviations,
- latent trajectory corrections.

9 Multi-Step Physics-Residual Trajectory Modeling

9.1 Motivation for Multi-Step Prediction

The original single-step framework predicts only a final future displacement:

$$x_{T+\Delta} = f(x_{1:T}) \tag{21}$$

Although this formulation can achieve reasonable endpoint accuracy, it suffers from several important limitations:

- the model only optimizes the final position,
- intermediate trajectory evolution is not constrained,
- Neural ODE dynamics may degenerate into a generic nonlinear mapping,

- temporal consistency of the predicted trajectory is not explicitly enforced.

From the perspective of ocean dynamics, marine drifter motion is fundamentally a continuous transport process rather than a single endpoint regression problem. Therefore, predicting only one future point does not fully exploit the advantages of continuous latent dynamics modeling.

To address these issues, the framework was extended from single-step prediction to multi-step trajectory forecasting.

9.2 Multi-Step Formulation

Instead of predicting only one future displacement, the upgraded model predicts an entire future trajectory sequence:

$$\hat{Y} = [\hat{x}_{T+1}, \hat{x}_{T+2}, \dots, \hat{x}_{T+K}] \quad (22)$$

where K denotes the prediction horizon.

The model now learns a trajectory evolution operator:

$$x_{t+1} = f_{\theta}(x_t) \quad (23)$$

rather than a single endpoint mapping.

This modification fundamentally changes the learning objective from:

- endpoint regression

to:

- continuous trajectory dynamics modeling.

10 Comparison Between Single-Step and Multi-Step Models

10.1 Overview of the Two Frameworks

The original framework uses a single-step prediction strategy, where the model predicts only one future displacement based on historical observations:

$$x_{T+\Delta} = f(x_{1:T}) \quad (24)$$

The upgraded framework extends the task into multi-step trajectory forecasting:

$$[x_{T+1}, x_{T+2}, \dots, x_{T+K}] = f(x_{1:T}) \quad (25)$$

Although both models share the same core components:

- Transformer encoder,
- Neural ODE latent dynamics,
- physics-residual learning,
- advection baseline correction,

their optimization objectives and dynamical behaviors differ substantially.

The single-step model mainly focuses on endpoint prediction accuracy, whereas the multi-step model explicitly learns continuous trajectory evolution.

11 Performance Comparison

The multi-step framework achieved the following evaluation results:

Metric	Value
RMSE Latitude	0.0591
RMSE Longitude	0.7425
Mean Geodesic Error	8.57 km
Median Geodesic Error	5.23 km
P90 Geodesic Error	14.96 km

Table 2: Performance of the Multi-Step Physics-Residual ODE Model

Compared with the previous single-step model, several important behavioral differences were observed.

11.1 Trajectory Consistency Improvement

The most important improvement of the multi-step framework is not necessarily the reduction of endpoint RMSE, but rather the enhancement of trajectory consistency.

The single-step model only constrains the final prediction point:

$$x_{T+\Delta} \tag{26}$$

As a result, intermediate latent evolution is weakly constrained. The model may produce physically unrealistic motion trajectories while still achieving acceptable endpoint errors.

In contrast, the multi-step model explicitly supervises the entire rollout trajectory:

$$x_{T+1}, x_{T+2}, \dots, x_{T+K} \tag{27}$$

This forces the latent dynamics to remain temporally coherent throughout the prediction horizon.

Consequently, the predicted trajectories become:

- smoother,
- more continuous,
- more physically realistic,
- dynamically more stable.

This behavior is particularly important for marine transport systems because ocean dynamics evolve continuously in time.

11.2 Improved Role of Neural ODE

One major limitation of the original single-step framework is that the Neural ODE module may effectively degenerate into a generic nonlinear hidden layer.

In the single-step setting, the latent state only needs to satisfy the final prediction objective:

$$z_0 \rightarrow z_T \quad (28)$$

However, in the multi-step framework, the latent trajectory must remain dynamically consistent over multiple rollout steps:

$$z(t_1), z(t_2), \dots, z(t_K) \quad (29)$$

This substantially increases the importance of continuous latent evolution modeling.

The Neural ODE therefore becomes more meaningful in the multi-step setting because it now models genuine temporal dynamics rather than merely performing nonlinear feature transformation.

The latent evolution equation:

$$\frac{dz(t)}{dt} = f_\theta(z(t), t) \quad (30)$$

begins to behave more like a continuous ocean transport simulator.

11.3 Residual Learning Analysis

The residual-learning mechanism also behaves differently in the two frameworks.

In the single-step model, the residual target is:

$$r = y_{future} - x_{adv} \quad (31)$$

Since the prediction horizon is relatively long, the residual often contains large nonlinear transport errors.

In the multi-step framework, the model instead performs sequential local corrections:

$$r_t = y_t - x_{adv,t} \quad (32)$$

This local residual formulation significantly reduces prediction difficulty because each correction only needs to compensate for short-term transport deviation.

As a result:

- optimization becomes more stable,
- rollout smoothness improves,
- local trajectory evolution becomes easier to learn.

12 Analysis of Evaluation Metrics

12.1 Mean and Median Geodesic Error

One particularly important observation is the difference between mean and median geodesic error:

$$\text{Mean Error} = 8.57 \text{ km} \quad (33)$$

$$\text{Median Error} = 5.23 \text{ km} \quad (34)$$

The relatively low median error indicates that the majority of predictions are reasonably accurate.

However, the larger mean error suggests the presence of several large-error outliers.

This indicates a heavy-tail error distribution, which is common in chaotic ocean transport systems.

The result implies that:

- most trajectories are predicted successfully,
- but a small number of trajectories experience substantial rollout drift.

12.2 P90 Geodesic Error

The P90 error is approximately:

$$14.96 \text{ km} \tag{35}$$

meaning that 90% of predicted trajectories remain within approximately 15 km geodesic error.

For long-horizon marine drift forecasting, this indicates relatively stable bulk prediction behavior.

The result suggests that the multi-step framework successfully stabilizes most trajectory rollouts.

12.3 Longitude Prediction Difficulty

A significant asymmetry exists between latitude and longitude prediction:

$$RMSE_{lat} = 0.059 \tag{36}$$

$$RMSE_{lon} = 0.742 \tag{37}$$

Longitude prediction is substantially more difficult.

Several factors contribute to this phenomenon.

First, longitude coordinates contain latitude-dependent scaling:

$$\Delta\lambda \propto \frac{1}{\cos(\phi)} \tag{38}$$

which introduces numerical instability at higher latitudes.

Second, zonal ocean transport is often more chaotic due to:

- mesoscale eddies,
- jet-like currents,
- turning flow structures,
- nonlinear circulation patterns.

Third, the current physics baseline:

$$x_{adv} = v\Delta t \tag{39}$$

assumes approximately constant velocity transport and cannot fully model turning flow or acceleration dynamics.

As a result, east-west transport errors accumulate more rapidly during rollout.

13 Limitations of the Current Multi-Step Framework

Although the multi-step framework improves trajectory consistency, several limitations remain.

13.1 Error Accumulation

Autoregressive rollout inevitably introduces cumulative error propagation:

$$\hat{x}_{t+1} \rightarrow \hat{x}_{t+2} \rightarrow \hat{x}_{t+3} \quad (40)$$

Small local prediction errors accumulate over time, eventually causing trajectory drift. This effect is particularly severe in chaotic ocean systems.

13.2 Weak Physics Baseline

The largest remaining bottleneck is likely the simplified advection prior:

$$x_{adv} = v\Delta t \quad (41)$$

This approximation assumes:

- constant velocity,
- no acceleration,
- no curvature,
- no evolving flow structure.

While residual learning compensates for part of the missing dynamics, long-horizon rollout still suffers from physics mismatch.

The results suggest that the current limitation is no longer representation learning capability, but rather insufficient physical prior modeling.